A VERY SMALL, VERY LOVELY WEB APP

Lovebug Booth

A tiny photo booth for two silly people.

PRODUCT BLUEPRINT · DESIGN SYSTEM · FEATURE SCOPE · IMPLEMENTATION PLAN



Approved visual concept: capture, Sticker Playground, and finished strip. Illustrative artwork, not real user photos.

PRIVATE, MOBILE-FIRST, $0-STARTER PROJECT

Design baseline · 11 October 2026 · Version 1.0

Product vision & scope

Lovebug Booth is a cozy photo-booth web app for a couple to take pictures, be ridiculous, decorate their memories, and keep the best ones. It must feel like a tiny shared scrapbook, not a social network or a commercial editing platform.

North star: she should be able to open it on her phone, take four photos, drag cute stickers around with her fingers, and save the result without confusion.

Non-negotiable product principles

- Just us: personal use; no marketplace, advertising, paid features, public profiles, comments, or follower system.
- Mobile first: design for a phone in portrait mode, then scale gracefully to tablets and laptops.
- The approved blush-pink scrapbook visual direction stays consistent across every screen.
- Capture and edit locally. No account, network connection, or cloud upload should be required to download a strip.
- The album is private and opt-in. Cloud saving will be built only after the local editing flow works.
- Use real, reusable sticker assets and self-made artwork; a Pinterest pin is inspiration, not proof of free licensing.
- Start at $0 with simple technology and no custom backend server.

Definition of a lovely first release

Two people can independently open the same HTTPS website. Each can choose a frame, use the phone camera or upload images, create a four-photo strip, decorate individual photos and the full strip, save an editable local draft, and export a clear image. We will add a private shared album in the next milestone.

Explicitly not building now

Payments, account administration, notifications, public discovery, public sharing feeds, paid sticker marketplaces, messaging, real-time multiplayer editing, or server-side AI image processing.

Approved look & visual references

The mood is Korean four-cut booth meets handmade scrapbook: pink gingham, bows, torn paper, shy little mascots, handwritten notes, and a little visual chaos. The interface itself should remain clean and readable; decorative clutter belongs around the photo, not over controls.



01 Captured strip



02 Sticker Playground



03 All done

These references show a visual target; they are not functioning screenshots. The final responsive editor will prioritize large touch controls and realistic photo preview sizes.

Visual system & interaction components

One coherent design system keeps every screen cute without making the application noisy. Each interactive surface should be easy to understand when the decorative artwork is removed.

Color, type & surface decisions

Role	Token	Example / use
Cream paper	#FFF8F4	App backgrounds and comfortable whitespace
Blush pink	#F7B9CA	Panels, borders, sticker tray, highlights
Cherry rose	#E77494	Primary buttons and active states
Butter yellow	#FFE5AE	Friendly accent and surprise micro-moments
Mint	#D6EEE3	Secondary sticker accents
Cocoa ink	#633C4B	Readable body text

Typography: Fraunces for romantic hero lettering; Nunito for readable application UI; Caveat used sparingly for handwritten captions. Use licensed or freely available fonts with dependable fallbacks.

UI building blocks

- Primary pink pill button: one clear action per screen, with readable text and generous tap area.
- Cream paper panels and soft white photo cards: a gentle sense of layered stationery, not heavy glass effects.
- Bottom sticker drawer on phones; anchored tool palette and persistent side panel on wide screens.
- Small outlined icons with text labels when meaning is not immediately obvious.
- Loading, empty, failed-camera, draft-restored and export-success states in the same friendly visual language.

Four frame themes to start

Classic Four-Cut: plain white/pink border; Strawberry Scrapbook: ribbon, lace and gingham; Silly Love Club: playful comic doodles; Soft Polaroid: warm cream and handwritten date stamps.

Motion, stickers and sound

Stickers may pop in with a tiny spring; buttons feel tactile; the final image can reveal like a developed strip. Keep effects modest around the camera, offer sound mute, and honor reduced-motion settings.

Navigation & complete screen list

One simple path: Welcome → Choose your vibe → Camera / upload → Review photos → Sticker Playground → Result → Optional private album. The path should not require signing in before the fun starts.

Screen	What the couple sees	Main actions
Welcome	Cute hero illustration, example strip, Start button, Album link	Start; view existing local creations
Choose a vibe	Frame/theme cards with live thumbnails, layout preview	Choose 4-cut strip, palette and countdown
Photo booth	Camera preview, shot counter, playful pose prompt	Allow camera, flip camera, capture, pause, cancel
Review shots	Four-photo strip preview; each cell can be retaken	Retake one; reorder; upload replacement
Sticker Playground	Canvas + floating controls + sticker drawer	Tap, drag, rotate, resize, edit, undo/redo
Finished strip	High-res preview, cute message, save/export actions	Download PNG/JPEG; save locally; cloud save later
Our little album	Private scrapbook grid of previous strips	Open, favorite, download, delete

Two valid journeys

Quick date-night mode Tap Start → four timed selfies → decorate → download. No login, no waiting, no obligation to save online.

Cozy scrapbook mode Import favorite gallery photos → assemble a strip → create custom cutouts → write captions → save an editable draft, then optionally add the final image to your private album.

UI copy should feel like the couple, not a tool

Examples: “oops, we’re adorable.” / “decorate our chaos” / “make it extra silly” / “our little album” / “save our masterpiece”. Avoid long instructions; show one context-sensitive hint when an action is new.

The Sticker Playground

This is the centerpiece. The editor is a true layered composition, not a flat filter over a camera screenshot. All elements remain movable until the user exports.



1. Tap an asset: add it centered in the visible composition.
2. Drag with one finger or the mouse: move accurately.
3. Pinch with two fingers or use corner handles: resize and rotate.
4. Tap to select: show a visible bounding box and friendly controls.
5. Undo/redo, duplicate, delete, front/back, flip, opacity.
6. Add text, handwriting, doodles and speech bubbles.
7. Decorate one photo or the entire strip; layers retain a clear scope.
8. Surprise Me: choose a sensible random sticker and place it safely.
9. Save an editable draft and export the exact visible composition.

Sticker drawer categories

Cute Pack (bows, hearts, flowers, animals, sparkles, cherries); Our Cutouts (user-made stickers); Words (romantic and funny caption presets); Doodles (pens, arrows, brush marks); Frames (tape, ribbons and paper decorations).

Critical usability rule: only the image canvas handles swipe/pinch editing. The sticker drawer, page scrolling, and Save buttons must remain easy to operate on a small screen.

Custom cutouts & sticker sourcing

How custom cutouts will work

1. Open “Us” → “Make a sticker” or select “Import sticker”.
2. Choose a photo from the device (or use a newly captured one).
3. Crop the subject. Use a simple manual keep/erase mask for background removal, with edge refinement.
4. Show a live transparent checkerboard preview and an optional white paper-sticker outline.
5. Name the sticker and save the result in the local “Our Sticker Box” for reuse.
6. Drop it onto any photo or completed strip; resize and rotate like a built-in sticker.

Cutout rollout

Initial build Import transparent PNG/WebP assets, plus simple crop and manual eraser/keep-mask cutouts entirely inside the browser.

Later enhancement Evaluate on-device automatic background removal only after checking mobile memory/performance and open-source licenses. Do not make an external upload a requirement.

Free visual asset sourcing rules

Source	Permitted workflow	Guardrail
Pinterest	Moodboard for colors, poses, compositions and sticker ideas	Do not copy random pins into the app without rights
Pixabay	Download applicable free assets and adapt into the visual system	Check asset license, exceptions, recognisable people and trademarks
OpenMoji	Adapt SVG emoji assets with required attribution/licence compliance	Check and follow the CC BY-SA conditions
Original assets	Draw your own bows, words, frames, mascots and cutout borders	Keep master editable artwork and exported transparent files
Your own photos	Create adorable couple-specific transparent cutouts	Only upload to cloud on a clear Save action

Asset folder rule

Each shared sticker has an ID, transparent file (SVG/PNG/WebP), preview, category, source URL, license/attribution text, and approximate display bounds. The project keeps this in a small manifest so the sticker picker and rights tracking stay manageable.

Mobile-first responsive design

The application is a web app, not a native Android/iOS build. It must still feel natural to use by touch. Build the narrowest layout first, then progressively enhance on larger screens.

Viewport	Navigation	Editor and canvas behavior
320–479 px	Compact header; bottom navigation if needed	Canvas above collapsible bottom sticker drawer; thumb-size controls; no horizontal page scroll
480–767 px	Same mobile journey with more breathing room	Larger canvas and 2-row sticker previews as space allows
768–1023 px	Tablet header + contextual controls	Canvas center, optional docked side panel; gestures still work
1024+ px	Desktop navigation with ample whitespace	Canvas center-left, persistent right-side sticker drawer and tools

Responsive interaction requirements

- Support Android Chrome, iPhone Safari, tablet browsers, and desktop Chrome/Edge with progressive enhancement.
- Respect notch/safe-area padding, dynamic browser toolbar height, orientation changes, and virtual keyboard.
- Keep essential touch targets about 44 × 44 CSS pixels; never hide a critical action behind hover-only behavior.
- Use pointer events and pointer capture for dragging. Disable touch scrolling only inside the editing surface.
- Scale screen preview to available space while storing element positions in normalized or original-strip coordinates.
- Preserve aspect ratio; export at a practical image resolution that does not depend on the phone screen size.
- Offer reduced-motion behavior and avoid large continuous effects while the camera/editor is active.
- Show gentle hints on first use (“drag to move”, “pinch to resize”) and don’t repeat them constantly.

Phone-specific acceptance checks

At 360 × 800 and 390 × 844 CSS px, there must be no clipped buttons, horizontal overflow, hidden confirmation dialogs, keyboard-covered text fields, or accidental scrolling while moving a sticker. Also verify landscape mode and a small 320 px viewport.

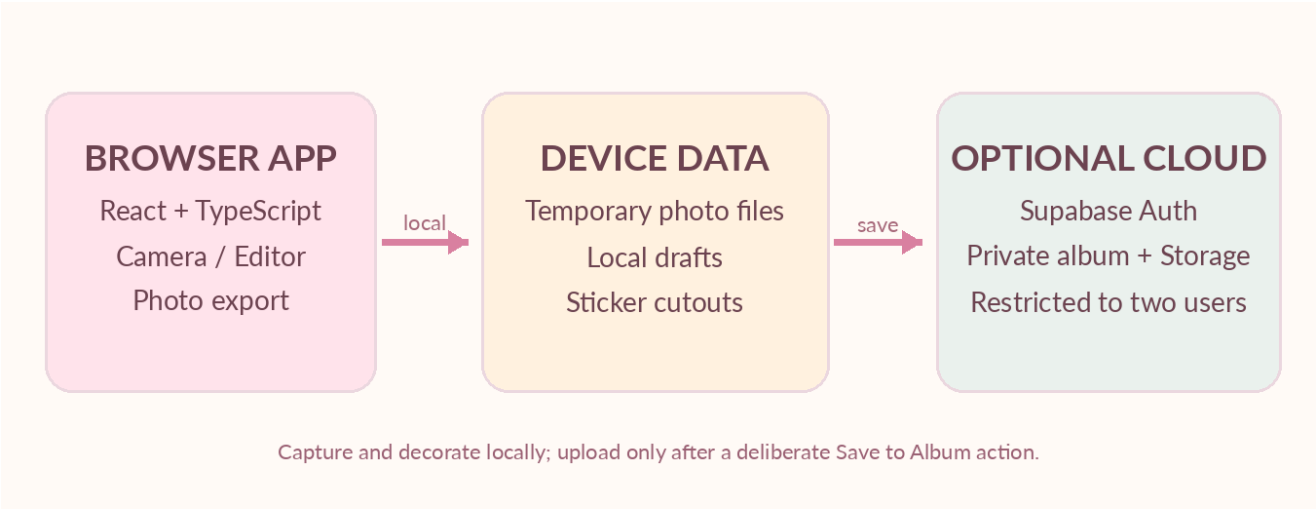
Fun features & meaningful behavior

Feature	Exact behavior	Delivery
Pose Roulette	A friendly random prompt before each photo; skip and reroll	Core
Photo capture	Default 4 shots, visible 3–2–1 countdown, shot progress	Core
Frame themes	Classic, Strawberry Scrapbook, Silly Love Club and Soft Polaroid	Core
Photo review	Retake a single slot or replace it with an upload	Core
Cute stickers	Tap, move, resize, rotate and change stacking order	Core
Custom cutouts	Upload a picture and make a transparent sticker	Core + polish
Text / doodles	Editable captions and direct doodling with a few brushes	Core + polish
Surprise Me	Place one random cute item without destroying composition	Polish
Filters	Small film/vintage/pastel selection; reversible	Polish
Undo / redo	Reversible edit history until export	Core
Local drafts	Leave and reopen an unfinished sticker project	Core
Export & share	Save PNG/JPEG; native share when browser supports it	Core
Our Little Album	Private two-user gallery with optional favorites	Cloud
Secret love notes	Attach a personal note to a saved photo strip	Later
Long-distance booth	Combine two separate sets of photos into a single strip	Later

Polish without performance loss

Playful button presses, subtle sticker pop-in, film developing reveal, and an optional camera shutter sound are welcome. No autoplay music; mute controls and reduced motion are first-class. Animations must never delay a required action or obscure the image.

Technical architecture



A simple local-first app; Supabase connects only when private sharing is enabled.

Recommended stack

Layer	Choice	Why
UI	React + TypeScript + Vite	Small, straightforward app; rapid iteration
Styling	Tailwind CSS + CSS variables	Responsive screens and reusable aesthetic tokens
Photo / sticker editor	Konva + react-konva	Editable object layers, transforms and canvas exports
Motion	CSS transitions + Motion only where helpful	Fun interactions without heavy rendering
Camera	Browser MediaDevices.getUserMedia	Live photo capture from mobile/desktop
Local persistence	IndexedDB	Editable drafts, local cutouts and session images
Cloud (later)	Supabase Auth, Postgres and private Storage	Couple-only shared collection without a custom server
Hosting	Cloudflare Pages via GitHub	Static deploys, HTTPS and deployment previews

Principles that keep implementation simple

- Prefer native browser capabilities before paid SDKs.
- Do not build Node/Express APIs, a custom server, queues or microservices.
- Keep capture, editing, flattening, and download client-side.
- Pin dependencies and document any editor-model or image-processing license before integration.

Photo composition & data lifecycle

End-to-end local flow

7. Camera (or uploads) yields four original images. Camera permissions are requested only when opening the booth.
8. Images are placed into a photo-strip template with fixed crop rules and optional filters.
9. The editor stores a structured document: frame choice, photo placements, stickers, text, doodles and layer order.
10. Draft state and custom local sticker files are saved in IndexedDB as appropriate; ephemeral object URLs are recreated when reopened.
11. Export renders the composition into PNG, or JPEG after flattening transparency; user explicitly downloads or shares the file.
12. Later, Save to Our Album uploads an optimized final image, thumbnail and metadata only after the user chooses cloud save.

Editor data contract (conceptual)

Field	Meaning
id, version	Unique draft ID and migration-friendly schema version
template / dimensions	Strip format and canonical export width/height
photos[4]	Photo asset IDs, crop transforms and filter settings
layers[]	Sticker/text/doodle objects with position, scale, rotation and z-order
customStickerIds[]	References to locally stored transparent cutout files
createdAt, updatedAt	Local timestamp metadata; no analytics tracking

Important editor invariants

Use stable original-space coordinates, not screen pixels; maintain original photos separately from compressed previews; never destructively overwrite images during an edit. Treat export as a deliberate render, not the project source. Keep undo history capped to prevent mobile memory spikes.

Supabase: optional private album

The app must be enjoyable without Supabase. We add cloud persistence only when local photo capture, sticker editing, and image export are solid. The public hosting site can be visible to anyone; the saved album must not be.

Small future schema

Table / resource	Essential fields and relationship
albums	id, title, created_by, created_at
album_members	album_id, user_id, role; only two authorized members
photo_strips	id, album_id, created_by, image_path, thumbnail_path, caption, created_at
saved_stickers (optional)	id, album_id, created_by, storage_path, name, created_at
Storage	Private bucket, per-album object paths, no public URLs

Security rules

- Supabase publishable key in frontend only. Never put a service-role/secret key inside a Vite browser build.
- Enable RLS on every exposed table; access is restricted by Auth user ID and album membership.
- Storage is a private bucket with authenticated, membership-scoped read/write/delete policies.
- Only the two allowlisted people should be accepted as members. A hidden UI link is not a security control.
- Do not upload raw camera captures automatically. Save only what was chosen, and support delete/export.
- Never publish unprotected permanent image URLs or store personal photographs in public GitHub repositories.

Authentication reality check

Supabase's default email sender is restricted and low-volume. We should choose the two-user login method when building the album: Google OAuth is a possible route; email sign-in needs working delivery for both addresses. Login does not block the local photo booth.

Free-tier expectations

At research time: Supabase Free includes 500 MB database size and 1 GB file storage. Projects may pause after low activity over a week; a paused project can be resumed via its dashboard. This is why local downloads and drafts stay independent of cloud availability.

Project folders & conventions

Use a single frontend repository. Avoid splitting into frontend/backend services for a two-person hobby app.

```
lovebug-booth/
  public/
    stickers/            # licensed/original visual assets
    icons/               # app/PWA icons
  src/
    app/                 # routing and application shell
    components/          # shared buttons, sheets, dialogs
    features/
      booth/              # camera, countdown, review
      editor/             # Konva stage, layers, history
      cutouts/            # import, mask and sticker box
      themes/             # frame layouts and palettes
      album/              # local gallery, cloud later
    lib/                 # image, storage, helpers
    styles/              # design tokens and globals
    types/               # editor and album contracts
  supabase/              # migrations and RLS (later)
  docs/                  # design reference and decisions
  .env.example
  README.md
```

Practical conventions

- One feature at a time, with small commits and visible progress.
- Keep colors, spacing, fonts and button variants in one shared design system.
- Store a sticker-license manifest alongside assets; do not rely on a screenshot of a Pinterest pin.
- No credentials committed to GitHub. .env.local is ignored; only public values have VITE_ prefix.
- Avoid dependencies until there is a concrete feature that needs them.

Implementation roadmap

Every milestone ends with something usable rather than a pile of infrastructure. We will keep the design approved and implement incrementally.

M0 — Foundation

Vite/React/TS repository, Tailwind tokens, typography, global shell, responsive navigation and photo-strip placeholders.

Done when: Phone-sized UI faithfully resembles the concept.

M1 — Real photo booth

HTTPS camera flow, 4-shot countdown, flip camera, permission/error handling, upload fallback, review and per-shot retake.

Done when: Capture four photos on her phone and see the strip.

M2 — Sticker Playground

Konva canvas, bundled stickers, transform handles, touch drag/pinch, object select, layers, undo/redo, text and PNG export.

Done when: Make a genuinely decorated downloadable strip.

M3 — Extra joy

Pose Roulette, theme selection, filters, doodles, Surprise Me, local drafts and custom cutouts/Sticker Box.

Done when: She can make personal reusable face stickers.

M4 — Private album

Supabase project, two-user sign-in decision, schema, RLS, private bucket, upload and album screens.

Done when: Both see only their own private shared album.

M5 — Release polish

PWA installability, offline-ready local editing, gentle animations, phone compatibility, safe image storage, accessibility.

Done when: Comfortable to use like an app on her phone.

Acceptance checklist & manual review

These are lightweight hands-on checks, not a heavyweight automated test suite. Run them at each milestone so we avoid building a cute but broken app.

Area	Must work before release
Mobile layout	320/360/390 px screens; no sideways scrolling; safe-area paddings; keyboard does not hide controls
Camera	Works over HTTPS after permission; clear denial/unavailable fallback; front/rear on supported hardware
Session	Exactly four saved photo slots; timer and retake keep correct ordering
Editor	Tap/drag/pinch/rotate; toolbar buttons work; no canvas drag scroll conflict
Cutouts	Import transparent PNG; crop/mask own photo; sticker reopens from local storage
Export	Download contains all decorations; correct proportions and usable resolution
Drafts	Refresh app and retrieve local saved draft when browser storage remains available
Privacy	No automatic photo upload; cloud images unavailable to nonmembers once album is enabled
Resilience	App remains usable without cloud access; handles low storage and interrupted sessions

Specific devices to check

- Android Chrome on your girlfriend's phone (primary acceptance device).
- Desktop Chrome or Edge for mouse interactions.
- iPhone Safari if available, especially download/share and camera permissions.
- Reduced-motion and low-power/slow-network conditions if convenient.

Common risks and guardrails

Mobile canvas memory → cap preview sizes and retain originals only as needed. Camera security → use HTTPS. Sticker attribution → keep source/license manifest. Cloud pause → local-first operation. Browser storage eviction → encourage users to download favorites; do not position a local draft as a permanent backup.

What to prepare before coding

No payment or production account is needed to start. Set up only what we will actually use in the first milestone.

01 Check Node.js and npm

Run node -v and npm -v. Use a supported current Node LTS; your existing Node 22 installation should be suitable for Vite.

02 Confirm Git is installed

Run git --version and make sure you can create a local Git repository.

03 Create a private GitHub repository

Name it lovebug-booth (or a preferred personal name). Leave it empty or with a README; do not commit couple photos or credentials.

04 Have VS Code (or preferred editor) ready

Install/use a TypeScript-friendly code editor and open the new project folder.

05 Keep the girlfriend's phone available

We will test the website from a deployed HTTPS preview once the camera module is implemented.

You do NOT need to configure Supabase, register a domain, pay for hosting, upload real personal photos, or collect Pinterest assets before we build the first screen.

First implementation command (when we begin)

```
npm create vite@latest lovebug-booth -- --template react-ts
cd lovebug-booth
npm install
npm run dev
```

Next task after setup: implement the homepage, responsive shell, design tokens, and the first photo-strip preview so the app immediately looks like our approved concept.

References & decisions

This plan separates visual inspiration from reusable assets and draws implementation constraints from official technical documentation. Re-check service plans and package versions when the related milestone begins.

Pinterest copyright and permissions
https://help.pinterest.com/en/article/copyright

Pixabay content license
https://pixabay.com/service/license-summary/

OpenMoji licensing
https://openmoji.org/

MDN browser camera (getUserMedia)
https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

MDN Pointer Events and touch handling
https://developer.mozilla.org/en-US/docs/Web/API/Pointer_events

Konva React canvas export
https://konvajs.org/docs/react/Canvas_Export.html

Supabase client security and RLS
https://supabase.com/docs/guides/database/secure-data

Supabase Storage RLS
https://supabase.com/docs/guides/storage/security/access-control

Supabase email restrictions
https://supabase.com/docs/guides/auth/auth-smtp

Supabase Free plan
https://supabase.com/pricing

Supabase free-project pausing
https://supabase.com/docs/guides/platform/free-project-pausing

Vite static deployment
https://vite.dev/guide/static-deploy.html

Settled choices

Scope: private couple application. Visual system: soft pink scrapbook. Priority: touch-friendly Sticker Playground and custom cutouts. Platform: responsive web application. Delivery: frontend-only first; Supabase private album when needed. Budget: $0 starter.

The concept art is an original illustrative mockup showing fictional placeholder couples. It is included to communicate the design and must not be mistaken for a built, tested product.